# Problem Set 7: Solutions

SPEA V706 Math Camp | Module 7 · Tuesday, August 11

## Part A. Concepts

### A1. Two dice

**a.** Each die has 6 faces and the dice are distinguishable, so $|\Omega| = 6 \times 6 = 36$. The dice are fair and the rolls are independent, so all 36 pairs are equally likely, each with probability 1/36.

**b.** The pairs summing to 7 are $(1,6), (2,5), (3,4), (4,3), (5,2), (6,1)$, six of them:

$$\Pr(\text{sum} = 7) = \frac{6}{36} = \frac{1}{6} \approx 0.167$$

**c.** Count the complement. There are $5 \times 5 = 25$ pairs with no 6, so

$$\Pr(\text{at least one 6}) = 1 - \frac{25}{36} = \frac{11}{36} \approx 0.306$$

Counting the complement is almost always easier than counting "at least one" directly.

**d.** Each die is even with probability 3/6, and the dice are independent:

$$\Pr(\text{both even}) = \frac{3}{6} \cdot \frac{3}{6} = \frac{9}{36} = \frac{1}{4}$$

**e.** Let $B$ be "at least one die shows a 3." By the same complement argument, $|B| = 36 - 25 = 11$. Inside $B$, the pairs that also sum to 7 are $(3,4)$ and $(4,3)$, so $|A \cap B| = 2$:

$$\Pr(\text{sum} = 7 \mid B) = \frac{2/36}{11/36} = \frac{2}{11} \approx 0.182$$

Conditioning shrank the sample space from 36 outcomes to 11. Learning that a 3 appeared makes a sum of 7 slightly more likely than it was unconditionally, because 3 is one of the faces that can be part of a 7.

### A2. A cross-tabulation

**a.** Pr(Insured) = 660/1000 = 0.66 and Pr(Employed) = 700/1000 = 0.70.

**b.** Restrict to the employed row, which has 700 people:

$$\Pr(\text{Insured} \mid \text{Employed}) = \frac{540}{700} \approx 0.771$$

**c.** Restrict to the insured column, which has 660 people:

$$\Pr(\text{Employed} \mid \text{Insured}) = \frac{540}{660} \approx 0.818$$

The first says: of employed people, 77% have insurance. The second says: of insured people, 82% are employed. Same cell in the numerator, different denominators, different questions. Reversing a conditional probability is the mistake Bayes' rule exists to prevent.

**d.** Independence requires $\Pr(I \cap E) = \Pr(I)\Pr(E)$. The left side is $540/1000 = 0.54$. The right side is $(0.66)(0.70) = 0.462$. They differ, so the two events are **not independent**. Equivalently, $\Pr(I \mid E) = 0.771 \neq 0.66 = \Pr(I)$: knowing someone is employed changes the chance they are insured, which in the United States is exactly what you would expect.

## A3. A flagged tax return

**a.** $\Pr(E) = 0.04$, $\Pr(F \mid E) = 0.90$, $\Pr(F \mid E^c) = 0.08$, and $\Pr(E^c) = 0.96$.

**b.** Split $F$ by whether the return has an error, which is the law of total probability:

$$\begin{aligned}\Pr(F) &= \Pr(F \mid E)\Pr(E) + \Pr(F \mid E^c)\Pr(E^c) \\ &= (0.90)(0.04) + (0.08)(0.96) \\ &= 0.036 + 0.0768 = 0.1128\end{aligned}$$

**c.** Now Bayes' rule:

$$\Pr(E \mid F) = \frac{\Pr(F \mid E)\Pr(E)}{\Pr(F)} = \frac{0.036}{0.1128} \approx 0.319$$

**d.** Only 4% of returns have errors, so the clean group is 24 times larger. Even though the flag misfires on only 8% of clean returns, 8% of a very large group (0.0768) outnumbers 90% of a very small one (0.036). About two-thirds of flags land on clean returns. A flag still raises the probability of an error from 4% to 32%, an eightfold increase, which is why the system is worth running even though most flags are false alarms.

## A4. Reading a CDF

**a.** Read it straight off the CDF: $\Pr(X \leq 2) = F(2) = 0.90$.

**b.** A jump in the CDF is the mass at that point: $\Pr(X = 2) = F(2) - F(1) = 0.90 - 0.55 = 0.35$.

**c.** $\Pr(1 < X \leq 3) = F(3) - F(1) = 1.00 - 0.55 = 0.45$. Note the strict inequality on the left, which is why we subtract $F(1)$ and not $F(0)$.

**d.** The median is the smallest $x$ with $F(x) \geq 0.5$. Here $F(0) = 0.10$ and $F(1) = 0.55$, so the **median is 1 car**. This is the quantile-as-inverse-CDF move: go in at probability 0.5 and come out at a value.

### A5. From an experiment to a random variable

**a.** $\Omega = \{HHH, HHT, HTH, THH, HTT, THT, TTH, TTT\}$, with $|\Omega| = 2^3 = 8$ equally likely outcomes.

**b.** $X$ counts the heads in each string: $X(HHH) = 3$; $X(HHT) = X(HTH) = X(THH) = 2$; $X(HTT) = X(THT) = X(TTH) = 1$; $X(TTT) = 0$. Notice that $X$ collapses eight outcomes into four values, which is the whole point of working with a random variable instead of $\Omega$.

**c.** Count how many outcomes map to each value and divide by 8:

| $x$ | 0 | 1 | 2 | 3 |
|---|---|---|---|---|
| $\Pr(X = x)$ | 1/8 | 3/8 | 3/8 | 1/8 |

**d.** The CDF accumulates that mass:

$$F_X(x) = \begin{cases} 0 & x < 0 \\ 1/8 & 0 \leq x < 1 \\ 4/8 & 1 \leq x < 2 \\ 7/8 & 2 \leq x < 3 \\ 1 & x \geq 3 \end{cases}$$

It is a staircase, flat between the integers and jumping at $0, 1, 2, 3$. Each jump is exactly the probability mass at that point: 1/8, 3/8, 3/8, 1/8. This is the discrete case from the deck, where the CDF has corners and no derivative exists.

## Part B. R

### B1. Check your setup

Any recent R (4.3 or later) is fine. `sessionInfo()` reports the version, the platform, and every package currently attached. The most common failure is a package that needs compilation; on macOS the fix is usually to install the Xcode command line tools, and on Windows to install Rtools.

### B2. R as a calculator, and assignment

```r
(17 + 4) / 3
```

```
[1] 7
```

```r
2^10
```

```
[1] 1024
```

```r
sqrt(144)
```

```
[1] 12
```

```r
n_outcomes <- 36
1 / n_outcomes
```

```
[1] 0.02777778
```

**c.** `<-` and `=` both assign, and `<-` is the convention in R and in this course. Reserve `=` for naming arguments inside a function call, as in `mean(x, na.rm = TRUE)`, so that reading a line tells you immediately which one is happening.

`x <- 5` assigns and returns nothing visible. `x == 5` is a **test**: it returns `TRUE` or `FALSE` and changes nothing. Writing `=` where you meant `==` inside an `if` is a classic bug, and R will usually error rather than silently do the wrong thing.

### B3. Vectors and their types

```r
rolls <- c(3, 1, 6, 6, 2, 4, 5, 6)

length(rolls)
```

```
[1] 8
```

```r
class(rolls)
```

```
[1] "numeric"
```

```r
mean(rolls)
```

```
[1] 4.125
```

```r
max(rolls)
```

```
[1] 6
```

```r
sum(rolls == 6)
```

```
[1] 3
```

**c.** A vector holds one type. `c(1, 2, "three")` cannot hold two numbers and a string at once, so R **coerces** everything to the most permissive type present, which is character:

```r
mixed <- c(1, 2, "three")
class(mixed)
```

```
[1] "character"
```

```r
mixed
```

```
[1] "1"     "2"     "three"
```

The numbers came back as `"1"` and `"2"`, in quotes. They are text now.

**d.** `mean(mixed)` warns that the argument is not numeric or logical and returns `NA`. R will not silently guess what you meant. This bites people when a stray text value in one cell of a spreadsheet column turns the whole imported column into character, and every summary of it comes back `NA`.

### B4. Indexing and logical subsetting

```r
rolls[1]                  # first
```

```
[1] 3
```

```r
rolls[length(rolls)]      # last
```

```
[1] 6
```

```r
rolls[2:4]                # a slice
```

```
[1] 1 6 6
```

```
rolls[rolls > 3]          # logical subsetting
```

```
[1] 6 6 4 5 6
```

```
rolls > 3                 # the logical vector itself
```

```
[1] FALSE FALSE  TRUE  TRUE FALSE  TRUE  TRUE  TRUE
```

```
mean(rolls > 3)           # the proportion
```

```
[1] 0.625
```

**c.** `rolls > 3` compares every element to 3 and returns a vector of `TRUE`/`FALSE` the same length as `rolls`. Putting that vector inside `[ ]` keeps the positions marked `TRUE` and drops the rest. Subsetting by a condition rather than by a position number is the move you will use constantly in `data.table`, where it becomes the `i` slot.

**d.** R stores `TRUE` as 1 and `FALSE` as 0, so the mean of a logical vector is the count of `TRUE`s over the total, which is a proportion. Here $5/8 = 0.625$. This is the same fact as the Bernoulli mean from the distributions deck: the mean of a 0/1 variable is a proportion.

### B5. Write a function

```
standardize <- function(x) {
  if (length(x) < 2) stop("standardize() needs at least two values")
  (x - mean(x)) / sd(x)
}

z <- standardize(rolls)
round(mean(z), 10)
```

```
[1] 0
```

```
sd(z)
```

```
[1] 1
```

**a.** The mean is 0 and the standard deviation is 1, by construction. Subtracting the mean centers the vector, and dividing by `sd` rescales it. This is exactly rule V2 from the deck, $\mathrm{Var}(aX + b) = a^2 \mathrm{Var}(X)$, with $b = -\bar{x}$ and $a = 1/s$. The mean prints as something like `-1.4e-17` rather than exactly `0`, which is floating point arithmetic, not an error.

**b.** The guard is the `if (...) stop(...)` line above. With fewer than two values `sd()` returns `NA` and the function would hand back a vector of `NA`s with no complaint. Failing loudly beats failing quietly.

### B6. Conditional probability on real data

```
pacman::p_load(data.table, wooldridge)
data(wage1, package = "wooldridge")
dt <- as.data.table(wage1)

nrow(dt)
```

```
[1] 526
```

```
head(names(dt), 10)
```

```
 [1] "wage"     "educ"     "exper"    "tenure"   "nonwhite" "female"
 [7] "married"  "numdep"   "smsa"     "northcen"
```

**a.** There are **526** workers. The variables include `wage` (hourly wage in 1976 dollars), `educ` (years of schooling), `exper` (years of potential experience), `female`, and `married`, among others.

**b, c.** The marginal, then the two conditionals:

```
dt[, mean(female)]                          # Pr(female = 1)
```

```
[1] 0.4790875
```

```
dt[female == 1, mean(married)]              # Pr(married | female)
```

```
[1] 0.5238095
```

```
dt[female == 0, mean(married)]              # Pr(married | male)
```

```
[1] 0.6861314
```

About 47.9% of the sample is female. Among women, about 52.4% are married; among men, about 68.6% are.

**d.** If `female` and `married` were independent, then Pr(married | female) and Pr(married | male) would both equal the overall marriage rate of 60.8%, and they would equal each other. They do not, differing by about 16 percentage points:

```
dt[, mean(married)]                                    # the marginal
```

```
[1] 0.608365
```

```r
dt[, .(pr_married = mean(married)), by = female]        # the two conditionals
```

```
   female pr_married
    <int>      <num>
1:      1  0.5238095
2:      0  0.6861314
```

So the two are not independent in this sample. Two cautions. First, this is a *sample*, so the two conditionals would differ a little even if the population probabilities were identical, and we do not yet have the tools to say whether a 16-point gap is more than noise. Day 4 starts building them. Second, the 1976 CPS extract is not a random sample of all adults, so read this as a fact about this dataset.

---

**Check yourself.**

1. An **outcome** $\omega$ is one result of the experiment. An **event** is a set of outcomes, so it is a subset of $\Omega$. A **random variable** is a function that assigns a number to each outcome. Outcomes and events live in $\Omega$; a random variable moves you to the real line, where you can do arithmetic.
2. Because $B$ becomes the new sample space. Conditioning throws away every outcome outside $B$, and the surviving probabilities have to be scaled up so they still sum to 1. Dividing by $\Pr(B)$ is that rescaling.
3. A vector holds a single type. Mixing a string in forces R to pick a type that can represent all three values, and only character can.